



HABIB UNIVERSITY

Data Structures & Algorithms

CS/CE 102/171 Spring 2025

Instructor: Maria Samad

Date: February 3rd, 2025

Quiz # 1

Time Allowed: 45 minutes

Max Points: 25

Name: _____

QUESTION 1: (20 marks)

You are given an input string called **str1** containing 'n' characters WITHOUT any blank spaces, where 'n' is at least 2. You are also given a queue called **QueueChars** of size 'n', which is initially empty. The program combines the repeating characters in the string in the same order as they appear in the given string, **str1** by following the steps as given below. Assume 'str1' is correctly given to you with the allowed length, so you may use it directly in your algorithm, without having to do any error checking or taking input. However, do declare it in the algorithm to define what it is, including the queues that you may use for your program.

DO NOT TRY TO ACHIEVE THE TASK IN ANY WAY OTHER THAN THE FOLLOWING STEPS:

1. It reads each character of string, **str1** and adds them to the queue, **QueueChars** one at a time, such that the first letter of the string appears at the front of the queue.
2. Once all letters are in the queue, you CANNOT use **str1** again anywhere in the program. If it is being used for any reason – simple or complex, the remaining algorithm will NOT be graded
3. Using the queue of letters, i.e. **QueueChars**, you should generate a new queue, called **QueueRepeat** that will contain all the letters combined together. If there is one such letter in the queue, it will appear once in the same order as it was defined in the original queue.

For example,

- Assume **str1** = "bippitybop", so as a result of first task, **QueueChars** = ['b', 'i', 'p', 'p', 'i', 't', 'y', 'b', 'o', 'p']
- Once **QueueChars** is filled, the resulting queue, **QueueRepeat** will contain all the letters combined together in the same order of their appearance. That is, for this example, **QueueRepeat** = ['b', 'b', 'i', 'i', 'p', 'p', 'p', 't', 'y', 'o']

You must implement the above tasks by writing an algorithm for each part.

Restrictions:

- Do not **HARD CODE** your design for the given example. The algorithm should work for any valid sized string.
- Assume the length of string is 'n', where $n > 1$, so there will always be at least two letters in any given string, resulting in Queues of size at least 2
- Assume None represents empty slots in all the data structures used for this question
- You are **NOT** allowed to use **any** additional ADT, including ListADT
- **DO NOT USE ANY OTHER DATA STRUCTURES INCLUDING ListADT/Python lists apart from the ones permitted to use**
- **REMEMBER:** Stacks and Queues are non-iterable and non-traversing data structures, so you CANNOT and MUST NOT do indexing within the data structures to access the elements

QUESTION 2: (5 marks)

Assume non-circular ListADT of size 4, with empty slots represented as None values in ADT. Fill the given table for the specified ListADT functions

Operation	Return Value	ListADT Contents
L.is_full()		
L.insert_last('A')		
L.insert_last('B')		
length(L)		
L.insert_first('C')		
L.remove_last()		
L.insert_last('T')		
L.is_full()		
L.remove_first()		
length(L)		
L.remove_first()		
L.remove_last()		
L.is_empty()		

